SC4051 - Distributed Systems

Lab Report

Demo Timeslot: 10th April 2026, 10:30-10:45

Group Member	Matric Number	Percentage of Work
Chen Xingwei	U2422151D	50%
Phua Wei Jie	U2422084G	50%

1. Introduction

This project implements a simple banking command-line interface application using a Client-Server architecture. The Client program provides a text-based interface for users to perform banking operations. The Server program maintains all bank account information in memory and executes the requested operations. The Server does not need to have persistent storage, and is assumed to handle requests single-threadedly.

Communication is done through UDP sockets, and all messages are marshalled and unmarshalled between the Client and Server as a sequence of bytes. In addition, the system has fault tolerance measures, and supports both At-Least-Once and At-Most-Once invocation semantics.

2. System Design

The Client is implemented in Go.

1. **main.go**: initialises a Client object/instance.
2. **requests.go**: serves as a proxy for handling request/reply messages, marshalling and unmarshalling data.
3. **rpc.go**: serves as the communication module, handling socket management and invocation logic.

The Server is implemented in Java.

1. **Server.java**: Main class that initialises and runs the Server.
2. **RequestHandler.java**: Controller class for handling different types of methods or command requests from the Client.
3. **AccountHandler.java**: Controller class for storing and handling all operations on Client accounts.
4. **Account.java**: Entity class for creating objects of Accounts with all of the necessary attributes (e.g., username, password).
5. **MonitorHandler.java**: Controller class for handling operations needed for satisfying the Client monitor functions (e.g., Registering, Callback).
6. **InvocationSemantic.java**: Base interface Class for the blueprint of the 2 invocation semantics (At-Least & At-Most Once).
7. **AtLeastOnce.java**: Controller class for handling raw requests from the Client.
8. **AtMostOnce.java**: Similar to AtLeastOnce Class, but has a hash table to store processed requests, to prevent executing duplicate requests.
9. **ClientInfo.java**: Entity class for storing the information of Clients that are currently monitoring.
10. **CurrencyType.java**: Enum for declaring all the currency types used in the bank
11. **ParseHandler.java**: Controller class for unmarshalling raw requests.
12. **UI.java**: Controller class for handling all the user interface in the Server side.

3. System Operations

We are required to implement 4 predetermined services, including 1 additional idempotent and 1 additional non-idempotent operation of our choosing. In addition, we have implemented a callback operation that allows Clients to express interest, and receive updates from the Server.

3.1 Create Account

Input: Client enters their *Account Name, Password, Currency Type, Initial Deposit*.
Output: Server creates an account and returns *Account Number*.

3.2 Close Account

Input: Client enters their *Account Name, Account Number, Password.*
Output: Server deletes the account and returns the status.

3.3 Deposit

Input: Client enters their *Account Name, Account Number, Password, Currency Type, Amount to Deposit.*
Output: Server updates the account balance for the currency selected and returns *Current Balance.*

3.4 Withdraw

Input: Client enters their *Account Name, Account Number, Password, Currency Type, Amount to Withdraw.*
Output: Server updates the account balance for the currency selected and returns *Current Balance.*

3.5 Additional Idempotent Operation: View Account Balance

Input: Client enters their *Account Name, Account Number, Password.*
Output: Server returns the *Currency: Balance* pairing for each currency.

This operation is idempotent as viewing the account balance multiple times does not change it.

3.6 Additional Non-Idempotent Operation: Transfer Money

Input: Client enters their *Account Name, Account Number, Password*, *Currency Type, Amount to Transfer, Target Account Number*.
Output: Server transfers the money to the target account and returns *Current Balance.*

This operation is non-idempotent as each successful transfer of money results in a change of the account state. Repeating the operation will result in multiple transfers, causing duplicate transfers and different account balances.

3.7 Callback

Input: Client enters *Time in Seconds* to register for callback.
Output: Server adds the account to the callback notification list for the amount of time stated.

4. Data Marshalling & Message Design

In this project, all Client requests are serialized into a custom message format before being sent over UDP. Each message first consists of a header indicating the request type, followed by a predetermined sequence of arguments required for the Server operation. Each field is prefixed with its length, ensuring that the Server can correctly parse each request and identify every argument, even when the fields vary in size.

4.1 Request from Client

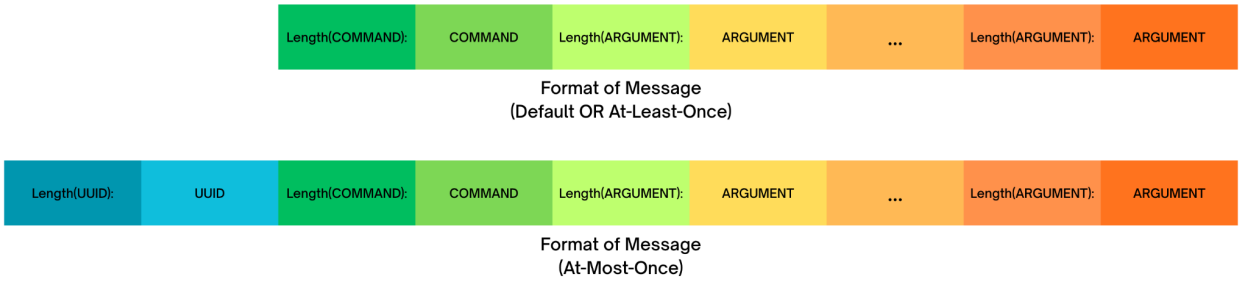

Fig 1. Request Message Format

For example, if we want to create an account, the request will be:

```
13:CREATEACCOUNT3:TOM4:12343:SGD5:10000
```

CREATEACCOUNT is the request type, *TOM* is the account name, *1234* is the password, *SGD* is the currency type, *10000* is the initial deposit.

```
_, err := conn.Write([]byte(request))
if err != nil {
    fmt.Println("Send error:", err)
    return "", err
}
```

The request message is then converted to a byte array before transmission.

4.2 Reply from Server

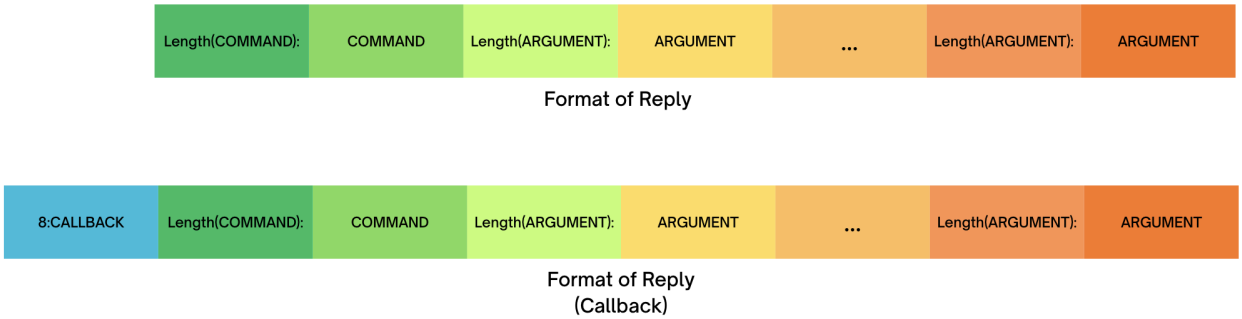

Fig 2. Reply Message Format

For example, if we successfully created an account, the reply will be:

```
20:CREATEACCOUNTSUCCESS4:1003
```

CREATEACCOUNTSUCCESS is the reply type, *1003* is the new account number.

4.3 Pros and Cons

Pros: The use of length-prefixed fields ensures that each argument can be clearly identified and extracted, even when the data contains delimiters such as colons.

Fields such as usernames, passwords, and currency values are flexible and can vary in size without affecting the correctness of parsing.

Cons: all inputs, including numerical values, are transmitted as strings, which increases packet size compared to more compact binary representations.

Additionally, unlike self-describing formats such as JSON, this design does not include field identifiers, making it less flexible and more tightly coupled between Client and Server.

5. Fault Tolerance

As packet loss is unlikely within a local area network, we simulate message loss in this project to test and demonstrate the fault-tolerance mechanisms.

5.1 Timeout

As UDP is an unreliable communication protocol, messages may be lost and fail to reach the Server or the Client. To handle this, the Client implements a timeout mechanism, where a request is retransmitted if no reply is received within a specified period. In this implementation, the timeout is set to 3000 ms, which serves as an upper bound on the time allowed for message transmission and operation execution. This ensures that communication failure is detected before a retransmission is sent.

In the defaultInvocation function, the timeout is handled using Go's **time.After**:

```
    select {
    case reply := <-replyChan:
        return reply, nil
    case <-time.After(TIMEOUT_MS * time.Millisecond):
        return "Timeout waiting for reply", fmt.Errorf("timeout")
    }
```

Here, the Client waits for a response from the Server via the reply channel. If no response arrives within the configured timeout, a timeout error is returned. This ensures that the Client does not block indefinitely due to network failures or Server unavailability.

5.2 Retransmission using Exponential Backoff

When a timeout occurs, the Client retransmits the request to increase the likelihood of successful delivery. This behavior is implemented in the atLeastOnce and atMostOnce functions, where a request may be resent up to a maximum of 10 retries. This limit prevents infinite retransmissions if the Server is unavailable.

To avoid overwhelming the Server or network with simultaneous retries from multiple Clients, retransmissions use exponential backoff. For each retry, the Client waits for a delay before retransmitting the request, with the delay doubling at each subsequent attempt.

```
    for i := 0; i < retries; i++ {

        reply, err := defaultInvocation(conn, request)
        if err == nil {
            return reply, nil
        }

        if i < retries-1 {
```

```
            delay := baseDelay * time.Duration(1<<i) // bit shift right, x2
            time.Sleep(delay)}}
```

By using exponential backoff, our implementation staggers Client retries, reducing the chance of simultaneous retransmissions. Although not implemented in this project, introducing a small random variation in the delay could also further decrease the chance of the Thundering Herd Problem.

6. Invocation Semantics

Invocation semantics define how a Client handles request delivery and retries in a distributed system. In this project, two main strategies are implemented: At-Least-Once and At-Most-Once.

Both strategies build upon defaultInvocation, which sends the request only once to the Server, without any retries. If the message is lost or the Server does not respond within the configured timeout, the Client receives an error and must manually resend the request.

The At-Least-Once and At-Most-Once strategies extend this basic behavior by automatically retransmitting requests to improve reliability and ensure correct delivery under network failures or packet loss.

6.1 At-Least-Once

6.1.1 Client

The At-Least-Once extends the default invocation by also implementing the fault tolerance measures as mentioned in the previous section. When the Client does not receive a reply after a set amount of time, it will retransmit the request using exponential backoff. If the Client receives a result, we can only guarantee if the operation has been executed at least once. This poses a problem if used for non-idempotent operations.

6.1.2 Server

The Server does not require any changes for At-Least-Once semantics and behaves the same as in the default invocation. It simply processes incoming requests and sends a reply once. Any retransmissions or retries are handled entirely by the Client.

6.2 At-Most-Once

6.2.1 Client

The At-Most-Once implementation is similar to the At-Least-Once implementation. The key difference is that each request is prefixed with an identifier before being sent to the Server.

```
    requestID := strconv.FormatInt(time.Now().UnixNano(), 10)

    fullRequest := fmt.Sprintf("%d:%s%s",
        len(requestID), requestID, request)
```

This allows the Server to detect and discard any duplicate requests, ensuring that each operation is executed at most once, even if retransmissions occur due to timeouts. If the Client receives a result, we can say that the operation has only been executed exactly once.

6.2.2 Server

The implementation for the Server will need to add a **hash table** that will **map** the request **UUID** received from the Client, to the **reply** that was sent to the Client. Therefore, anytime the Client retries due to reasons like the reply being lost midway, the Server will see that this request has already been executed once, so it will just send the saved reply, instead of executing the same request again and replying to the Client.

7. Experiment & Analysis

We used a random number generator to simulate packet loss.

```
    if rand.Float64() < PACKET_LOSS_PROBABILITY {
        fmt.Println("Simulating packet loss for request:", request)
        return "", fmt.Errorf("Simulated packet loss.")
    }
```

To evaluate the correctness of the invocation semantics, experiments were conducted by simulating packet loss and observing the outputs. The experiments and results are categorised into two groups, idempotent and non-idempotent operations, allowing for a clearer comparison of how each invocation strategy behaves under different conditions.

7.1 Idempotent Operations

Idempotent operations include *close account, view balance.* An example is shown for *view balance*.

At-least-once

```
--- Choose Invocation Semantics: ---
1. Default
2. At-Least-Once
3. At-Most-Once
Select: 2

--- MAIN MENU: type 'exit' to quit ---
1. Create Account
2. Delete Account
3. Deposit
4. Withdraw
5. View Balance
6. Transfer
7. Register for Updates
8. Exit
Select: 5
Account Name: tom
Account Number: 1000
Password: 123

Initial request sent.
Retransmission 1 will be sent in 100ms

Retransmission 1 sent.
Retransmission 2 will be sent in 200ms

Retransmission 2 sent.
Retransmission 3 will be sent in 400ms

Retransmission 3 sent.
Retransmission 4 will be sent in 800ms

Retransmission 4 sent.
Retransmission 5 will be sent in 1.6s

Retransmission 5 sent.
Current Balance:
SGD 50.0
```

```
BOOTING UP...
Select Invocation Semantics:
  1) At-Least-Once
  2) At-Most-Once
Enter Number: 1
Running At-Least-Once Invocation Semantics

^^^^^^^Server running on port 2222...^^^^^^^
Received: 4:VIEW3:tom4:10003:123 Request from: /10.91.253.13 port64828
!!!!!!Packet Loss!!!!!!
Received: 4:VIEW3:tom4:10003:123 Request from: /10.91.253.13 port64828
!!!!!!Packet Loss!!!!!!
Received: 4:VIEW3:tom4:10003:123 Request from: /10.91.253.13 port64828
!!!!!!Packet Loss!!!!!!
Received: 4:VIEW3:tom4:10003:123 Request from: /10.91.253.13 port64828
!!!!!!Packet Loss!!!!!!
Received: 4:VIEW3:tom4:10003:123 Request from: /10.91.253.13 port64828
!!!!!!Packet Loss!!!!!!
Received: 4:VIEW3:tom4:10003:123 Request from: /10.91.253.13 port64828
```

Left: *Client*, Right: *Server*

Despite multiple communication failures, idempotent operations still result in the same output, as multiple executions of the same operation do not change the final output.

At-most-once

For the same reasons, idempotent operations also function correctly on at-most-once semantics.

7.2 Non-Idempotent Operations

Non-idempotent operations include *create account, deposit*, *withdraw*, *transfer.* To prevent repetition, we only show an example for *deposit.* However, the same behaviour is seen among the other non-idempotent operations too.

7.2.1 Deposit

At-least-once

```
Current Balance:
SGD 50.0

--- MAIN MENU: type 'exit' to quit ---
1. Create Account
2. Delete Account
3. Deposit
4. Withdraw
5. View Balance
6. Transfer
7. Register for Updates
8. Exit
Select: 3
Account Name: tom
Account Number: 1000
Password: 123
Select Currency Type:
1: SGD
2: EUR
3: JPY
4: USD
Choice: 1
Amount to Deposit: 100

Initial request sent.
Retransmission 1 will be sent in 100ms

Retransmission 1 sent.
Deposit Successful.
New Balance (SGD): 250.0
```

We first view the current balance of the account. The current balance is SGD 50.0

Next, we select the option to deposit SGD into the same account. We deposit SGD 100.0

The initial request is sent along with a retransmission. The final output shows the account balance as SGD 250.0, when it should be 150.0!

Client

Checking the server reveals that the initial response was executed on the server, but the reply from the server to the client was lost. This resulted in the retransmission from the client, causing the server to mistakenly execute the deposit again, effectively depositing twice. The execution of the deposit twice changed the result as it is a non-idempotent operation.

```
^^^^^^^Server running on port 2222...^^^^^^^
Received: 4:VIEW3:tom4:10003:123 Request from: /10.91.253.13 port63029
Received: 7:DEPOSIT3:tom4:10003:1233:SGD3:100 Request from: /10.91.253.13 port63029
!!!!!!Packet Loss!!!!!!
Received: 7:DEPOSIT3:tom4:10003:1233:SGD3:100 Request from: /10.91.253.13 port63029
```

Server

At-most-once

The same operations were conducted again, but with at-most-once semantics. Despite multiple communication failures and retransmissions, the final output was the correct amount of SGD 150.0 .

```
Current Balance:
SGD 50.0

--- MAIN MENU: type 'exit' to quit ---
1. Create Account
2. Delete Account
3. Deposit
4. Withdraw
5. View Balance
6. Transfer
7. Register for Updates
8. Exit
Select: 3
Account Name: tom
Account Number: 1000
Password: 123
Select Currency Type:
1: SGD
2: EUR
3: JPY
4: USD
Choice: 1
Amount to Deposit: 100
DEBUG: 19:1775118098158342000 7:DEPOSIT3:tom4:10003:1233:SGD3:100

Initial request sent.
Retransmission 1 will be sent in 100ms

Retransmission 1 sent.
Retransmission 2 will be sent in 200ms

Retransmission 2 sent.
Retransmission 3 will be sent in 400ms

Retransmission 3 sent.
Retransmission 4 will be sent in 800ms

Retransmission 4 sent.
Deposit Successful.
New Balance (SGD): 150.0
```

Client

With at-most-once semantics, each request has its own unique UUID. The server correctly detects that it is receiving the same duplicate request from the client, and does not execute the deposit operation again. The deposit is executed only once.

```
^^^^^^^Server running on port 2222...^^^^^^^
Received: 19:1775118082075238000 4:VIEW3:tom4:10003:123 from: /10.91.253.13 port57152
Received: 19:1775118098158342000 7:DEPOSIT3:tom4:10003:1233:SGD3:100 from: /10.91.253.13 port57152
!!!!!!Packet Loss!!!!!!
Received: 19:1775118098158342000 7:DEPOSIT3:tom4:10003:1233:SGD3:100 from: /10.91.253.13 port57152
Duplicate detected for UUID: 1775118098158342000
!!!!!!Packet Loss!!!!!!
Received: 19:1775118098158342000 7:DEPOSIT3:tom4:10003:1233:SGD3:100 from: /10.91.253.13 port57152
Duplicate detected for UUID: 1775118098158342000
!!!!!!Packet Loss!!!!!!
Received: 19:1775118098158342000 7:DEPOSIT3:tom4:10003:1233:SGD3:100 from: /10.91.253.13 port57152
Duplicate detected for UUID: 1775118098158342000
!!!!!!Packet Loss!!!!!!
Received: 19:1775118098158342000 7:DEPOSIT3:tom4:10003:1233:SGD3:100 from: /10.91.253.13 port57152
Duplicate detected for UUID: 1775118098158342000
```

Server